# CAREER ADVANCEMENT SKILLS

## ASSIGNMENT-2

**Name: Subhapreet Patro**
**Roll No.: 2211CS010547**
**Section: I**

**1.Application Configuration Manager, You are tasked with developing a Java-based configuration manager for a simple desktop application. The application requires loading, reading, and updating configuration settings such as username, theme, auto-save interval, and language. These settings should be stored in a .properties file and managed using Java's Properties class.**

**Solution:**

**1. Sample config.properties File**

```
username=guest
theme=light
autoSaveInterval=5
language=en
```

**2. ConfigManager.java - Java Code**

```
import java.io.*;
import java.util.Properties;
import java.util.Scanner;

public class ConfigManager {
    private final String configFilePath;
    private final Properties properties;

    public ConfigManager(String configFilePath) {
        this.configFilePath = configFilePath;
        this.properties = new Properties();
    }

    // Load properties from file
    public void loadConfig() throws IOException {
        File file = new File(configFilePath);
        if (!file.exists()) {
            System.out.println("Configuration file not found. Creating new one.");
            saveDefaultConfig();
        }
```

```
    try (FileInputStream fis = new FileInputStream(configFilePath)) {
        properties.load(fis);
    }
}

// Save properties to file
public void saveConfig() throws IOException {
    try (FileOutputStream fos = new FileOutputStream(configFilePath)) {
        properties.store(fos, "Application Configuration");
    }
}

// Save default config if file not found
private void saveDefaultConfig() throws IOException {
    properties.setProperty("username", "guest");
    properties.setProperty("theme", "light");
    properties.setProperty("autoSaveInterval", "5");
    properties.setProperty("language", "en");
    saveConfig();
}

// Getter method
public String getSetting(String key) {
    return properties.getProperty(key);
}

// Setter method
public void updateSetting(String key, String value) {
    properties.setProperty(key, value);
}

// Display current settings
public void printSettings() {
    System.out.println("Current Configuration:");
    for (String key : properties.stringPropertyNames()) {
        System.out.println(key + " = " + properties.getProperty(key));
    }
}

// Main method for demonstration
public static void main(String[] args) {
    String path = "config.properties";
```

```
        ConfigManager manager = new ConfigManager(path);
        Scanner scanner = new Scanner(System.in);

        try {
            manager.loadConfig();
            manager.printSettings();

            System.out.println("\nDo you want to update a setting? (yes/no)");
            if (scanner.nextLine().equalsIgnoreCase("yes")) {
                System.out.print("Enter setting key: ");
                String key = scanner.nextLine();
                System.out.print("Enter new value: ");
                String value = scanner.nextLine();
                manager.updateSetting(key, value);
                manager.saveConfig();
                System.out.println("Configuration updated successfully.\n");
                manager.printSettings();
            }
        } catch (IOException e) {
            System.err.println("Error managing configuration: " + e.getMessage());
        }
    }
}
```

**Output:**

**First Run (when config.properties doesn't exist):**

```
Configuration file not found. Creating new one.
Current Configuration:
username = guest
theme = light
autoSaveInterval = 5
language = en

Do you want to update a setting? (yes/no)
yes
Enter setting key: theme
Enter new value: dark
Configuration updated successfully.

Current Configuration:
username = guest
theme = dark
```

autoSaveInterval = 5
language = en

**File Output (config.properties after update)**
#Application Configuration
#Mon Jul 28 10:02:34 IST 2025
username=guest
theme=dark
autoSaveInterval=5
language=en

```
C:\Users\sairam\OneDrive\Desktop\379>java ConfigManager
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
[0.049s][warning][cds] A jar file is not the one used while building the shared archive file: C:\Users\
sairam\AppData\Roaming\Code\User\globalStorage\pleiades.java-extension-pack-jdk\java\21\lib\modules
[0.049s][warning][cds] A jar file is not the one used while building the shared archive file: C:\Users\
sairam\AppData\Roaming\Code\User\globalStorage\pleiades.java-extension-pack-jdk\java\21\lib\modules
[0.050s][warning][cds] C:\Users\sairam\AppData\Roaming\Code\User\globalStorage\pleiades.java-extension-
pack-jdk\java\21\lib\modules timestamp has changed.
Current Configuration:
autoSaveInterval = 5
theme = light
language = en
username = guest

Do you want to update a setting? (yes/no)
yes
Enter setting key: theme
Enter new value: dark
Configuration updated successfully.

Current Configuration:
autoSaveInterval = 5
theme = dark
language = en
username = guest
```

## 2.How to implement collection ordering?

**Solution:**

### 1. Using TreeSet (for Sorted Sets)

- Automatically sorts elements in **natural order** or using a **custom comparator**.

**Example:**

```
import java.util.*;
public class TreeSetExample {
    public static void main(String[] args) {
```

```
        System.out.println("Subhapreet Patro: 2211CS010547");
        Set<String> names = new TreeSet<>();
        names.add("John");
        names.add("Alice");
        names.add("Bob");
        for (String name : names) {
            System.out.println(name); // Output: Alice, Bob, John (sorted)
        }
    }
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Alice
Bob
John
```

**3.Word Frequency Counter (Map & Set)**

**Write a Java program that reads a paragraph of text and prints the frequency of each unique word in descending order of frequency.**

**Requirements:**

- **Ignore case and punctuation.**
- **Use HashMap to count words.**
- **Use TreeMap or sorting logic to sort by frequency.**
- **Use Set to handle unique words.**

**Solution:**

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Scanner;
import java.util.Set;
```

```
public class WordFrequencyCounter {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Subhapreet Patro: 2211CS010547");
        System.out.println("Enter a paragraph:");
        String input = scanner.useDelimiter("\\A").next(); // Read full input

        // Step 1: Clean text - remove punctuation and convert to lowercase
        String cleanedText = input.replaceAll("[^a-zA-Z ]", "").toLowerCase();

        // Step 2: Split into words
        String[] words = cleanedText.split("\\s+");

        // Step 3: Count word frequency using HashMap
        Map<String, Integer> wordCountMap = new HashMap<>();
        Set<String> uniqueWords = new HashSet<>(); // for unique word set

        for (String word : words) {
            uniqueWords.add(word);
            wordCountMap.put(word, wordCountMap.getOrDefault(word, 0) + 1);
        }

        // Step 4: Sort by frequency in descending order using List and Comparator
        List<Map.Entry<String, Integer>> sortedList = new
ArrayList<>(wordCountMap.entrySet());

        sortedList.sort((a, b) -> b.getValue().compareTo(a.getValue())); // descending order
```

```
        // Step 5: Print results
        System.out.println("\nWord Frequencies (Descending):");
        for (Map.Entry<String, Integer> entry : sortedList) {
            System.out.println(entry.getKey() + " = " + entry.getValue());
        }

        // Step 6: Optional - Print unique words
        System.out.println("\nUnique Words (" + uniqueWords.size() + "):");
        for (String word : uniqueWords) {
            System.out.print(word + " ");
        }
    }
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Enter a paragraph:
Java is a powerful language. Java is used everywhere! Is Java difficult? No,
    Java is simple.
Word Frequencies (Descending):
java = 4
is = 3
a = 1
powerful = 1
language = 1
used = 1
everywhere = 1
difficult = 1
no = 1
simple = 1

Unique Words (10):
a simple language used everywhere java no is powerful difficult
```

**4.Remove Duplicates and Sort (List & Set)**
**Objective:**
**Given a list of integers with duplicates, write a Java program to:**
1. **Remove duplicates.**
2. **Sort the result in ascending order.**

**Requirements:**
- **Use List to accept input.**
- **Use Set (e.g., LinkedHashSet or TreeSet) to remove duplicates.**
- **Return a sorted list.**

**Input: [4, 2, 7, 2, 4, 9, 1]**
**Output: [1, 2, 4, 7, 9]**

**Solution:**

```
import java.util.*;
public class RemoveDuplicatesAndSort {
   public static void main(String[] args) {
      // Step 1: Input list with duplicates
      System.out.println("Subhapreet Patro: 2211CS010547");
      List<Integer> inputList = Arrays.asList(4, 2, 7, 2, 4, 9, 1);
      System.out.println("Original List: " + inputList);
      // Step 2: Remove duplicates using TreeSet (automatically sorted)
      Set<Integer> uniqueSortedSet = new TreeSet<>(inputList);
      // Step 3: Convert back to List (if needed)
      List<Integer> sortedList = new ArrayList<>(uniqueSortedSet);
      // Step 4: Output result
      System.out.println("Sorted List without Duplicates: " + sortedList);
   }
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Original List: [4, 2, 7, 2, 4, 9, 1]
Sorted List without Duplicates: [1, 2, 4, 7, 9]
```

**5.Employee Directory (Map & Custom Class)**
**Objective:**
**Create a program that stores and displays an employee directory using a Map where:**
- **The key is the department name.**
- **The value is a list of employee names in that department.**

**Requirements:**
- **Use HashMap<String, List<String>>.**

- **Provide options to add, search, and list employees by department.**

**Solution:**

```
import java.util.*;

public class EmployeeDirectory {
   // Map to store department and employee list
   private final Map<String, List<String>> directory = new HashMap<>();
   private final Scanner scanner = new Scanner(System.in);

   public static void main(String[] args) {
      System.out.println("Subhapreet Patro: 2211CS010547");
      EmployeeDirectory ed = new EmployeeDirectory();
      ed.runMenu();
   }

   // Main menu
   public void runMenu() {
      int choice;
      do {
         System.out.println("\n--- Employee Directory ---");
         System.out.println("1. Add Employee");
         System.out.println("2. Search by Department");
         System.out.println("3. List All Departments and Employees");
         System.out.println("4. Exit");
         System.out.print("Choose an option: ");
         choice = scanner.nextInt();
         scanner.nextLine(); // consume newline

         switch (choice) {
            case 1 -> addEmployee();
            case 2 -> searchByDepartment();
            case 3 -> listAll();
            case 4 -> System.out.println("Exiting...");
            default -> System.out.println("Invalid choice! Try again.");
         }
      } while (choice != 4);
   }

   // Add employee to a department
   private void addEmployee() {
      System.out.print("Enter department name: ");
      String department = scanner.nextLine().trim();
```

```
        System.out.print("Enter employee name: ");
        String employee = scanner.nextLine().trim();

        // If department doesn't exist, create a new list
        directory.putIfAbsent(department, new ArrayList<>());
        directory.get(department).add(employee);

        System.out.println("Employee added successfully.");
    }

    // Search and display employees in a department
    private void searchByDepartment() {
        System.out.print("Enter department name to search: ");
        String department = scanner.nextLine().trim();

        List<String> employees = directory.get(department);
        if (employees == null || employees.isEmpty()) {
            System.out.println("No employees found in this department.");
        } else {
            System.out.println("Employees in " + department + ":");
            for (String emp : employees) {
                System.out.println("- " + emp);
            }
        }
    }

    // List all departments and their employees
    private void listAll() {
        if (directory.isEmpty()) {
            System.out.println("Directory is empty.");
            return;
        }

        System.out.println("\nAll Departments and Employees:");
        for (Map.Entry<String, List<String>> entry : directory.entrySet()) {
            System.out.println("\nDepartment: " + entry.getKey());
            for (String emp : entry.getValue()) {
                System.out.println(" - " + emp);
            }
        }
    }
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
--- Employee Directory ---
1. Add Employee
2. Search by Department
3. List All Departments and Employees
4. Exit
Choose an option: 1
Enter department name: HR
Enter employee name: Subhapreet Patro
Employee added successfully.
--- Employee Directory ---
1. Add Employee
2. Search by Department
3. List All Departments and Employees
4. Exit
Choose an option: 4
Exiting...
```

**6.Design a real-time data handling system where:**

- **Sensors (Producers) continuously generate data and push it into a shared data queue (buffer).**

**A Processing Unit (Consumer) fetches data from the buffer for real-time processing and action**

**Solution:**

```
import java.util.Random;
import java.util.concurrent.*;

// Sensor (Producer)
class Sensor implements Runnable {
    private final BlockingQueue<Integer> dataQueue;
    private final String sensorName;
    private final Random random = new Random();

    public Sensor(String sensorName, BlockingQueue<Integer> queue) {
        this.sensorName = sensorName;
        this.dataQueue = queue;
    }

    @Override
    public void run() {
        while (true) {
            try {
                int data = random.nextInt(100); // Simulated sensor data
                dataQueue.put(data); // Blocking put
                System.out.println(sensorName + " produced data: " + data);
```

```
                Thread.sleep(500 + random.nextInt(500)); // Simulate interval
            } catch (InterruptedException e) {
                System.out.println(sensorName + " interrupted.");
                break;
            }
        }
    }
}

// Processing Unit (Consumer)
class ProcessingUnit implements Runnable {
    private final BlockingQueue<Integer> dataQueue;

    public ProcessingUnit(BlockingQueue<Integer> queue) {
        this.dataQueue = queue;
    }

    @Override
    public void run() {
        while (true) {
            try {
                int data = dataQueue.take(); // Blocking take
                System.out.println("Processing Unit consumed data: " + data);
                process(data);
            } catch (InterruptedException e) {
                System.out.println("Processing Unit interrupted.");
                break;
            }
        }
    }

    private void process(int data) {
        // Simulated processing (can add ML, logging, alerts, etc.)
        if (data > 80) {
            System.out.println(">> ALERT: High value detected! (" + data + ")");
        }
    }
}

// Main class
public class RealTimeDataSystem {
    public static void main(String[] args) {
```

```
        BlockingQueue<Integer> dataQueue = new LinkedBlockingQueue<>(10); // Shared buffer

        Thread sensor1 = new Thread(new Sensor("Sensor-1", dataQueue));
        Thread sensor2 = new Thread(new Sensor("Sensor-2", dataQueue));
        Thread processor = new Thread(new ProcessingUnit(dataQueue));

        // Start threads
        sensor1.start();
        sensor2.start();
        processor.start();

        // Run for a fixed time (optional), then stop
        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            sensor1.interrupt();
            sensor2.interrupt();
            processor.interrupt();
            System.out.println("System shutdown initiated...");
        }));
    }
}
```

**Output:**

```
>> ALERT: High value detected! (90)
Sensor-1 produced data: 33
Processing Unit consumed data: 33
Sensor-2 produced data: 42
Processing Unit consumed data: 42
Sensor-1 produced data: 72
Processing Unit consumed data: 72
Sensor-2 produced data: 14
Processing Unit consumed data: 14
Sensor-1 produced data: 20
Processing Unit consumed data: 20
Sensor-2 produced data: 61
Processing Unit consumed data: 61
Sensor-1 produced data: 51
Processing Unit consumed data: 51
Sensor-2 produced data: 97
Processing Unit consumed data: 97
```

**7.Online Food Delivery System Using Multithreading**

**In an online food delivery system, multiple customers place orders concurrently, and multiple delivery agents process and deliver those orders. Each order must be picked up from the restaurant and delivered to the customer.**

**Design a multithreaded system that simulates:**

- **Multiple customers (threads) placing food orders.**
- **Restaurants (shared resources) preparing food.**
- **Delivery agents (threads) picking up and delivering orders.**

**Solution:**

```
import java.util.*;
import java.util.concurrent.*;

// Represents a food order
class Order {
    private static int idCounter = 1;
    private final int orderId;
    private final String customerName;
    private final String foodItem;

    public Order(String customerName, String foodItem) {
```

```
        this.orderId = idCounter++;
        this.customerName = customerName;
        this.foodItem = foodItem;
    }

    public int getOrderId() { return orderId; }
    public String getCustomerName() { return customerName; }
    public String getFoodItem() { return foodItem; }

    @Override
    public String toString() {
        return "Order #" + orderId + " (" + foodItem + ") for " + customerName;
    }
}

// Customer thread placing order
class Customer implements Runnable {
    private final BlockingQueue<Order> orderQueue;
    private final String customerName;
    private final List<String> menu = List.of("Pizza", "Burger", "Sushi", "Pasta", "Tacos");

    public Customer(String name, BlockingQueue<Order> queue) {
        this.customerName = name;
        this.orderQueue = queue;
    }
    @Override
    public void run() {
        try {
            Random rand = new Random();
            String food = menu.get(rand.nextInt(menu.size()));
            Order order = new Order(customerName, food);
            orderQueue.put(order); // Place order
            System.out.println("[Customer] " + customerName + " placed " + order);
        } catch (InterruptedException e) {
            System.out.println("[Customer] " + customerName + " was interrupted.");
        }
    }
}

// Restaurant that prepares the food
class Restaurant {
    public void prepareOrder(Order order) throws InterruptedException {
        System.out.println("[Restaurant] Preparing " + order);
```

```
        Thread.sleep(1000); // Simulate time to prepare food
    }
}

// Delivery Agent thread
class DeliveryAgent implements Runnable {
    private final BlockingQueue<Order> orderQueue;
    private final Restaurant restaurant;
    private final String agentName;

    public DeliveryAgent(String name, BlockingQueue<Order> queue, Restaurant
restaurant) {
        this.agentName = name;
        this.orderQueue = queue;
        this.restaurant = restaurant;
    }

    @Override
    public void run() {
        while (true) {
            try {
                Order order = orderQueue.take(); // Pick order from queue
                restaurant.prepareOrder(order);
                System.out.println("[DeliveryAgent] " + agentName + " picked up " + order);
                Thread.sleep(1000); // Simulate delivery time
                System.out.println("[DeliveryAgent] " + agentName + " delivered " + order);
            } catch (InterruptedException e) {
                System.out.println("[DeliveryAgent] " + agentName + " was interrupted.");
                break;
            }
        }
    }
}

// Main class
public class FoodDeliverySystem {
    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        BlockingQueue<Order> orderQueue = new LinkedBlockingQueue<>(10);
        Restaurant restaurant = new Restaurant();

        // Start delivery agents
        Thread agent1 = new Thread(new DeliveryAgent("Agent-1", orderQueue, restaurant));
```

```
        Thread agent2 = new Thread(new DeliveryAgent("Agent-2", orderQueue, restaurant));
        agent1.start();
        agent2.start();

        // Start customer threads
        for (int i = 1; i <= 5; i++) {
            Thread customer = new Thread(new Customer("Customer-" + i, orderQueue));
            customer.start();
            try {
                Thread.sleep(700); // Delay between customers
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }

        // Shutdown logic after demo (optional)
        new Timer().schedule(new TimerTask() {
            public void run() {
                agent1.interrupt();
                agent2.interrupt();
                System.out.println("System shutdown...");
            }
        }, 10000); // Stop after 10 seconds
    }
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
[Restaurant] Preparing Order #1 (Sushi) for Customer-1
[Customer] Customer-1 placed Order #1 (Sushi) for Customer-1
[Customer] Customer-2 placed Order #2 (Tacos) for Customer-2
[Restaurant] Preparing Order #2 (Tacos) for Customer-2
[DeliveryAgent] Agent-1 picked up Order #1 (Sushi) for Customer-1
[Customer] Customer-3 placed Order #3 (Burger) for Customer-3
[DeliveryAgent] Agent-2 picked up Order #2 (Tacos) for Customer-2
[DeliveryAgent] Agent-1 delivered Order #1 (Sushi) for Customer-1
[Restaurant] Preparing Order #3 (Burger) for Customer-3
[Customer] Customer-4 placed Order #4 (Tacos) for Customer-4
[DeliveryAgent] Agent-2 delivered Order #2 (Tacos) for Customer-2
[Restaurant] Preparing Order #4 (Tacos) for Customer-4
[Customer] Customer-5 placed Order #5 (Tacos) for Customer-5
[DeliveryAgent] Agent-1 picked up Order #3 (Burger) for Customer-3
[DeliveryAgent] Agent-2 picked up Order #4 (Tacos) for Customer-4
[DeliveryAgent] Agent-1 delivered Order #3 (Burger) for Customer-3
[Restaurant] Preparing Order #5 (Tacos) for Customer-5
[DeliveryAgent] Agent-2 delivered Order #4 (Tacos) for Customer-4
```

## 8. Remove duplicates from Array

**Solution:**

```
import java.util.*;

public class RemoveDuplicates {
   public static void main(String[] args) {
      System.out.println("Subhapreet Patro: 2211CS010547");
      int[] arr = {4, 2, 7, 2, 4, 9, 1};

      // Step 1: Convert array to List
      List<Integer> list = new ArrayList<>();
      for (int num : arr) {
         list.add(num);
      }

      // Step 2: Use Set to remove duplicates
      Set<Integer> set = new LinkedHashSet<>(list); // maintains insertion order

      // Step 3: Convert back to array (optional)
      int[] uniqueArray = set.stream().mapToInt(Integer::intValue).toArray();

      // Output
      System.out.println("Original Array: " + Arrays.toString(arr));
      System.out.println("Array after removing duplicates: " +
Arrays.toString(uniqueArray));
   }
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Original Array: [4, 2, 7, 2, 4, 9, 1]
Array after removing duplicates: [4, 2, 7, 9, 1]
```

## 9. How to sort an array of objects

**Hint:- Camparable & comparator interfaces**

**Solution:**

```
import java.util.*;

class Student implements Comparable<Student> {
   String name;
   int marks;
```

```
    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }

    @Override
    public int compareTo(Student s) {
        return this.marks - s.marks; // Ascending order
    }

    @Override
    public String toString() {
        return name + " - " + marks;
    }
}

public class SortUsingComparable {
    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        Student[] students = {
            new Student("Alice", 85),
            new Student("Bob", 70),
            new Student("Charlie", 92)
        };

        Arrays.sort(students); // Uses compareTo()

        for (Student s : students) {
            System.out.println(s);
        }
    }
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Bob - 70
Alice - 85
Charlie - 92
```

## 10. Find Higest & lowest Frequency elements of an array

**Solution:**

```
import java.util.*;

public class FrequencyFinder {
   public static void main(String[] args) {
      System.out.println("Subhapreet Patro: 2211CS010547");
      int[] arr = {2, 3, 4, 2, 3, 3, 1, 5, 2, 5};
      Map<Integer, Integer> freqMap = new HashMap<>();
      for (int num : arr) {
         freqMap.put(num, freqMap.getOrDefault(num, 0) + 1);
      }
      int maxFreq = Collections.max(freqMap.values());
      int minFreq = Collections.min(freqMap.values());
      List<Integer> maxFreqElements = new ArrayList<>();
      List<Integer> minFreqElements = new ArrayList<>();

      for (Map.Entry<Integer, Integer> entry : freqMap.entrySet()) {
         if (entry.getValue() == maxFreq) {
            maxFreqElements.add(entry.getKey());
         }
         if (entry.getValue() == minFreq) {
            minFreqElements.add(entry.getKey());
         }
      }
      System.out.println("Original Array: " + Arrays.toString(arr));
      System.out.println("Frequency Map: " + freqMap);
      System.out.println("Highest Frequency: " + maxFreq + ", Elements: " +
maxFreqElements);
      System.out.println("Lowest Frequency: " + minFreq + ", Elements: " +
minFreqElements);
   }
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Original Array: [2, 3, 4, 2, 3, 3, 1, 5, 2, 5]
Frequency Map: {1=1, 2=3, 3=3, 4=1, 5=2}
Highest Frequency: 3, Elements: [2, 3]
Lowest Frequency: 1, Elements: [1, 4]
```